

TCP Connection Management and Flow Control

Module 11

Duration: 20:27

Video: <https://www.youtube.com/watch?v=TDYgEmXtWHc>

Course Coordinator - Dr Maninder Singh, Professor and Dean, Academic Affairs, Thapar University, Patiala

Subject Expert - Sunil Kumar, Department of Electrical and Computer Engineering, Punjab Agriculture University, Ludhiana

Auto-generated YouTube transcript for personal study. Spoken wording belongs to the original lecture. Follow the video link above; this file is not an official CEC/SWAYAM publication.

0:00 [Music]

0:16 Hello friends I welcome you all to the video lecture series of cyber security and information security in this lecture

0:24 we will study about TCP connection management and flow control first of all please the contents which I shall

0:31 cover after covering brief introduction about TCP connection establishment I shall take up data transfer in TCP and

0:40 last I shall take up TCP connection termination so now let us start TCP is a connection oriented

0:49 transport protocol establishes a virtual path between the source and destination all of these segments

0:56 belonging to a message are then sent over this virtual path using a single virtual pathway for the

1:03 entire message facilitates the acknowledgement process as well as the retransmission of damaged or lost frames

1:12 you may wonder how TCP which uses the services of Ip a connectionless protocol can be connection

1:20 oriented the point is that a TCP connection is virtual not physical TCP operates at a higher level TCP uses the

1:29 services of Ip to deliver individual segments to the receiver but it controls the connection

1:35 itself if a segment is lost or corrupted it is retransmitted unlike TCP IP is unaware of this

1:45 retransmission if a segment arrives out of order TCP holds it until the missing

1:51 segments arrive IP is unaware of this reordering in TCP connection oriented

1:58 transmission requires three phases connection establishment data transfer and connection

2:07 termination connection establishment TCP transmits data in full duplex mode when two tcps in two

2:16 machines are connected then they are able to send segments to each other simultaneously this implies that each

2:23 party must initialize communication and get approval from the other party before any data are

2:31 transferred three-way hand checking the connection establishment in TCP is called three-way handshaking in our

2:40 example an application program called the client wants to make a connection with another application program called

2:48 the server using TCP as the transport layer protocol the process starts with the

2:54 server the server program tells its TCP that it is ready to accept a connection

3:01 3 minutes, 1 second this request is called a passive open although the server TCP is ready to accept a connection from any machine in

3:10 the world it cannot make the connection itself the client program issues a request for an active open a client that

3:19 wishes to connect to an open server tells its TCP to connect to a particular server TCP can now start the three-way

3:29 handshaking process as shown in the figure to show the process we use

3:36 timelines each segment has values for all its header fields and perhaps for some of its option fields too however we

3:46 show only the few fields necessary to understand each phase we show the sequence number the acknowledgement

3:53 number the control flags and window size if relevant

4:00 the three steps in this pH are as follows the client sends the first segment a sync segment in which only the

4:09 sync flag is set this segment is for synchronization of sequence numbers the client in our example

4:18 chooses a random number as the first sequence number and sends this number to the server this sequence number is called

4:27 the initial sequence number is S note that this segment does not contain any acknowledgement number it does not

4:36 define the window size either a window size definition makes sense only when a segment includes an

4:44 acknowledgement the segment can also include some options that we discuss later in the chapter note that the S YN

4:52 segment is a control segment that carries no data however it consumes one sequence

4:59 number when the data transfer starts the Isn is incremented by one we can say

5:06 that the S YN segment carries no real data but we can think of it as containing one imaginary bite a sync

5:15 segment cannot carry data but it consumes one sequence number the server sends the second segment a s YN plus a

5:25 segment with two flag bits set s YN and a this segment has the Dual Purpose

5:34 first it is a s YN segment for communication in the other direction the server uses this segment to initialize

5:42 the sequence number for numbering the bytes sent from the server to the client the server also acknowledges the receipt

5:50 of the SN segment from the client by setting the a flag and displaying the next sequence number it expects to

6:00 receive from the client because it contains an acknowledgement it also needs to define the receive window size

6:09 rent as we will see in the flow control section the client sends the third

6:15 segment this is just an a segment it acknowledges the receipt of the second

6:22 segment with the AK flag and analment number field noted the sequence number

6:29 in this segment is the same as the one in the S YN segment the AK segment does not consume any sequence numbers the

6:37 client must also Define the server window size some implementations allow this third segment in the connection

6:45 phase to carry the first chunk of data from the client in this case the third segment must have a new sequence number

6:53 showing the bite number of the first bite in the data in general the third segment usually does not carry data and

7:02 consumes no sequence numbers simultaneous open a rare situation may occur when

7:10 both processes issue an active open in this case both tcps transmit a SN plus a

7:18 segment to each other and one single connection is established between them S1 flooding

7:26 attack the connection establishment procedure in TCP is susceptible to a serious security problem called Sy and

7:36 flooding attack this happens when one or more malicious attackers send a large number of s segments to a server

7:45 pretending that each of them is coming from a different client by faking The Source IP addresses in the datagrams the

7:53 server assuming that the clients are issuing an active open allocates the necessary sources such as creating

8:01 8 minutes, 1 second transfer control block TCB tables and setting timers the TCP server then sends

8:08 the SN plus AK segments to the fake clients which are lost when the server waits for the Third Leg of the hand

8:17 shaking process H sources are allocated without being used if during this short period of time the number of SN segments

8:26 is large the server eventually Runs Out of resources and may be unable to accept connection request from valid

8:36 clients this is y and flooding attack belongs to a group of security attacks known as a denial of service attack in

8:45 which an attacker monopolizes a system with so many service request that the system overloads and denies service to

8:54 valid requests some implementations of TCP have strategies to to alleviate the

9:00 effect of syn attack some have imposed a limit of connection requests during a specified period of time others try to

9:09 filter out datagrams coming from unwanted Source addresses one recent strategy is to postpone resource allocation until the

9:18 server can verify that the connection request is coming from a valid IP address by using what is called a

9:27 cookie data transfer after connection is established BYOD directional data transfer can take place

9:35 the client and server can send data and acknowledgements in both directions the acknowledgement is piggybacked with the

9:42 data figure shows an example in this example after connection is established the client sends 2,000 bytes of data in

9:51 two segments the server then sends 2,000 bytes in one segment the client sends one more segment the first three

10:00 segments carry both data and acknowledgement but the last segment carries only an acknowledgement because there is no more data to be sent note

10:10 the values of the sequence and acknowledgement numbers the data segment sent by the client have the psh flag set

10:18 so that the server TCP tries to deliver data to the server process as soon as they are received we discuss the use of

10:27 this flag in more detail later the segment from the server on the other hand does not set the push flag most TCP

10:37 implementations have the option to set or not set this flag pushing data we saw that the

10:45 sending TCP uses a buffer to store the stream of data coming from the sending application program the sending TCP can

10:54 select the segment size the receiving TCP also buffers the data when they arrive and delivers them

11:02 to the application program when the application program is ready or when it is convenient for the receiving

11:09 TCP this type of flexibility increases the efficiency of TCP however there are occasions in which the application

11:17 program has no need for this flexibility for example consider an application program that communicates interactively

11:25 with another application program on the other end the application program on one site wants to send a key stroke to the

11:33 application at the other site and receive an immediate response delayed transmission and delayed delivery of

11:40 data may not be acceptable by the application program TCP can handle such a situation

11:48 the application program at the center can request a push operation this means that the sending TCP must not wait for

11:56 the window to be filled it must create a segment and send it immediately the sending TCP must also set the push bit

12:05 to let the receiving TCP know that the segment includes data that must be delivered to the receiving application

12:12 program as soon as possible and not to wait for more data to come although the push operation can be requested by the

12:20 application program most current TCP implementations ignore such requests TCP

12:27 can choose whether or not to use this feature urgent data TCP is a stream oriented protocol

12:37 this means that the data is presented from the application program to TCP as a stream of bytes each bite of data has a

12:45 position in the Stream however there are occasions in which an application program needs to send urgent bytes some

12:53 bites set need to be treated in a special way by the application at the other end the solution is to send a

13:00 segment with the urg bit set the sending application program tells the sending

13:06 TCP that the piece of data is urgent the sending TCP creates a segment and inserts the Urgent data at the beginning

13:16 of the segment the rest of the segment can contain normal data from the buffer the Urgent pointer field in the header

13:24 defines the end of urgent data when they receiving TC P receives a segment with the urg bit set it informs

13:33 the receiving application of the situation how this is done depends on the operating system it is then to

13:42 discretion of the receiving program to take an action it is important to mention that

13:49 tcp's urgent data is neither a priority service nor an expedited data service

13:56 rather TCP urgent mode is a service by which the application program at the center side marks some portion of the

14:05 bite stream as needing special treatment by the application program at the receiver site thus signaling the

14:15 presence of urgent data and marking its position in the data stream are the only aspects that distinguish the delivery of

14:23 urgent data from the delivery of all other TCP data for all other purposes urgent data is treated

14:32 identically to the rest of the TCP by stream the application program at the receiver site must read every bite of

14:40 data exactly in the order it was submitted regardless of whether or not urgent mode is used the standard TCP as

14:49 implemented does not ever deliver any data out of order connection

14:56 termination any of the two parties involved in exchanging data either client or either server can close the

15:03 connection although it is usually initiated by the client most implementations today allow

15:10 two options for connection termination three-way handshaking and four-way handshaking with the half close option

15:19 three-way handshaking most implementations today allow three-way handshaking for connection termination as shown in the

15:28 figure as per figure I'll explain the working in three steps first in a common situation the

15:36 client TCP after receiving a close command from the client process sends the first segment a f segment in which

15:45 the F flag is set note that a f segment can include the last chunk of data

15:51 sent by the client or it can be just a control segment as shown in the figure if it is only a control segment it

16:00 consumes only one sequence number the F segment consumes one sequence number if

16:07 it does not carry data second the server TCP after

16:14 receiving the F segment informs its process of the situation and sends the second segment a f plus a segment to

16:23 confirm the receipt of the F segment from the client and at the same time to announce the closing of the connection in the other

16:31 direction this segment can also contain the last chunk of data from the server if it does not carry data it consumes

16:40 only one sequence number third the client TP sends the last segment and a segment to confirm

16:49 the receipt of the F segment from the TCP server this segment contains the acknowledgement number which is 1 plus

16:57 the sequence number received in the FY segment from the server this segment cannot carry data

17:04 and consumes no sequence numbers half close in TCP one and can

17:11 stop sending data while still receiving data this is called a half close either the server or the client can issue a

17:20 half close request it can occur when the server needs all the data before processing can begin a good example is

17:29 sorting when the client sends data to the server to be sorted the server sends to receive all the data before sorting

17:38 can start this means the client after sending all data can close the connection in the client to server

17:46 Direction however the server to client direction must remain open to return the sorted data the server after receiving

17:55 the data still needs time for sorting its outbound direction must remain open figure shows an example of an half

18:04 close in the figure data transfer from the client to the server stops the client half closes The Connection by

18:12 sending a f segment the server accepts the half close by sending the AK

18:20 segment the server however can still send data when the server has sent all of the processed data it sends a f i

18:29 segment which is acknowledged by an a from the client after half closing the connection data can travel from the

18:38 server to the client and acknowledgments can travel from the client to the server the client cannot send any more data to

18:47 the server note the sequence numbers we have used the second segment consumes no

18:54 sequence number although the client has received sequence number y less than equal to 1 the server sequence number is

19:03 still y greater than equal to 1 when the connection finally closes the sequence number of the last AK segment is still X

19:13 because no sequence numbers are consumed during data transfer in that direction so friends this was all about

19:22 TCP connection management and flow control hope the concepts explained in this lecture were understanding able and

19:29 helpful hope to see you in the next lecture till then goodbye thank you

19:38 [Music]

19:45 [Music]